

SKILL-10

Session-19:

Task-1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_count = 0;

/* Signal handler */
void signal_handler(int sig, siginfo_t *info, void *context)
{
    (void)context;

    signal_count++;

    printf("\n-----\n");

    switch (sig)
    {
        case SIGINT:
            printf("[Handler] SIGINT received.\n");
            printf("[Handler] Interrupt signal handled.\n");
            break;

        case SIGTERM:
            printf("[Handler] SIGTERM received.\n");
            printf("[Handler] Termination request handled.\n");
            break;

        case SIGUSR1:
            printf("[Handler] SIGUSR1 received.\n");
            printf("[Handler] User-defined event 1 handled.\n");
            break;

        case SIGUSR2:
            printf("[Handler] SIGUSR2 received.\n");
            printf("[Handler] User-defined event 2 handled.\n");
```

```

if (sigaction(SIGINT, &action, NULL) == -1)
{
    perror("sigaction SIGINT");
    return 1;
}

/* Register SIGTERM */
if (sigaction(SIGTERM, &action, NULL) == -1)
{
    perror("sigaction SIGTERM");
    return 1;
}

/* Register SIGUSR1 */
if (sigaction(SIGUSR1, &action, NULL) == -1)
{
    perror("sigaction SIGUSR1");
    return 1;
}

/* Register SIGUSR2 */
if (sigaction(SIGUSR2, &action, NULL) == -1)
{
    perror("sigaction SIGUSR2");
    return 1;
}

printf("\nFour signal handlers registered successfully.\n");

printf("\nSignals configured:\n");
printf("1. SIGINT - Interrupt\n");
printf("2. SIGTERM - Termination request\n");
printf("3. SIGUSR1 - User-defined signal 1\n");
printf("4. SIGUSR2 - User-defined signal 2\n");

printf("\nWaiting for signal events...\n");
printf("Press Ctrl+C to test SIGINT.\n");

```

```

printf("\nSignals configured:\n");
printf("1. SIGINT - Interrupt\n");
printf("2. SIGTERM - Termination request\n");
printf("3. SIGUSR1 - User-defined signal 1\n");
printf("4. SIGUSR2 - User-defined signal 2\n");

printf("\nWaiting for signal events...\n");
printf("Press Ctrl+C to test SIGINT.\n");

/* Keep program running */
while (1)
{
    pause();

    printf("\n[Main] Process remains stable after signal handling.\n");
    printf("[Main] Waiting for next signal...\n");
}

return 0;

```

```

=====
      SIGNAL HANDLER MANAGEMENT
=====
Process PID: 1510

Four signal handlers registered successfully.

Signals configured:
1. SIGINT - Interrupt
2. SIGTERM - Termination request
3. SIGUSR1 - User-defined signal 1
4. SIGUSR2 - User-defined signal 2

Waiting for signal events...
Press Ctrl+C to test SIGINT.
^C
-----
[Handler] SIGINT received.
[Handler] Interrupt signal handled.
Signal number : 2
Sender PID    : 0
Total signals : 1
-----

[Main] Process remains stable after signal handling.
[Main] Waiting for next signal...
^C
-----
[Handler] SIGINT received.
[Handler] Interrupt signal handled.
Signal number : 2
Sender PID    : 0
Total signals : 2
-----

[Main] Process remains stable after signal handling.
[Main] Waiting for next signal...
^Z
[2]+  Stopped                  ./skill10s19task1
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully registered handlers for four signal events using `sigaction()`. Signal actions were configured with `SA_SIGINFO` to obtain signal context information such as the signal number and sender PID. `SIGINT`, `SIGTERM`, `SIGUSR1`, and `SIGUSR2` were tested individually, and the corresponding handlers processed each event without terminating the program. The process remained stable after handling the signals and continued waiting for subsequent events using `pause()`. Thus, signal-handler registration, signal-action configuration, interrupt handling, signal-context management, stability verification, and testing of four signal events were successfully demonstrated.

Task-2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

volatile sig_atomic_t foreground_pid = -1;

/* SIGINT handler for the shell */
void sigint_handler(int sig)
{
    (void)sig;

    printf("\n[Shell] SIGINT (Ctrl+C) captured.\n");

    if (foreground_pid > 0)
    {
        printf("[Shell] Forwarding SIGINT to foreground job PID %d.\n",
            foreground_pid);

        kill(foreground_pid, SIGINT);
    }
    else
    {
        printf("[Shell] No foreground job is running.\n");
    }
}

int main()
{
    pid_t pid;
    int status;

    /* Protect the shell from SIGINT */
    struct sigaction action;

    action.sa_handler = sigint_handler;
```

```

/* Parent acts as shell */
foreground_pid = pid;

printf("\n[Shell] Foreground job PID: %d\n", pid);
printf("[Shell] Job state: RUNNING\n");
printf("\nPress Ctrl+C to test interrupt handling.\n");

/* Wait for foreground job */
waitpid(pid, &status, 0);

/* Foreground job has ended */
foreground_pid = -1;

if (WIFSIGNALED(status))
{
    printf("\n[Shell] Foreground job terminated by signal %d.\n",
        WTERMSIG(status));

    printf("[Shell] Job state: TERMINATED\n");
}
else if (WIFEXITED(status))
{
    printf("\n[Shell] Foreground job exited normally.\n");
    printf("[Shell] Job state: COMPLETED\n");
}

printf("\n[Shell] Shell process is still running.\n");
printf("[Shell] SIGINT protection is active.\n");

printf("\n===== \n");
printf("          INTERRUPT HANDLING COMPLETE\n");
printf("===== \n");

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ ./skill10s19task2
=====
FOREGROUND INTERRUPT HANDLER
=====
[Shell] Shell PID: 1693
[Shell] SIGINT handler installed.

[Shell] Foreground job PID: 1694
[Shell] Job state: RUNNING

Press Ctrl+C to test interrupt handling.

[Job] Foreground job started.
[Job] PID: 1694
[Job] Running... Press Ctrl+C to interrupt.
[Job] Working...
[Job] Working...
[Job] Working...
[Job] Working...
[Job] Working...
^C
[Shell] SIGINT (Ctrl+C) captured.
[Shell] Forwarding SIGINT to foreground job PID 1694.

[Shell] Foreground job exited normally.
[Shell] Job state: COMPLETED

[Shell] Shell process is still running.
[Shell] SIGINT protection is active.

=====
INTERRUPT HANDLING COMPLETE
=====
ruhi0404@Ruhitha:~/Operating_Systems$ ps -ef | grep skill10s19task2
ruhi0404      1712      312  0 04:02 pts/0    00:00:00 grep --color=auto skill1
0s19task2
ruhi0404@Ruhitha:~/Operating_Systems$ pgrep -a skill10s19task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill10s19task2
=====
FOREGROUND INTERRUPT HANDLER
=====

```

```

=====
ruhi0404@Ruhitha:~/Operating_Systems$ ps -ef | grep skill10s19task2
ruhi0404    1712      312  0 04:02 pts/0    00:00:00 grep --color=auto skill1
0s19task2
ruhi0404@Ruhitha:~/Operating_Systems$ pgrep -a skill10s19task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill10s19task2
=====
                FOREGROUND INTERRUPT HANDLER
=====
[Shell] Shell PID: 1734
[Shell] SIGINT handler installed.

[Shell] Foreground job PID: 1735
[Job] Foreground job started.
[Shell] Job state: RUNNING

Press Ctrl+C to test interrupt handling.
[Job] PID: 1735
[Job] Running... Press Ctrl+C to interrupt.
[Job] Working...
[Job] Working...
[Job] Working...
[Job] Working...
[Job] Working...
[Job] Working...
[Job] Working...
[Job] Working...
^C
[Shell] SIGINT (Ctrl+C) captured.
[Shell] Forwarding SIGINT to foreground job PID 1735.

[Shell] Foreground job exited normally.
[Shell] Job state: COMPLETED

[Shell] Shell process is still running.
[Shell] SIGINT protection is active.

=====
                INTERRUPT HANDLING COMPLETE
=====
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully captured SIGINT generated by Ctrl+C without terminating the shell process. The shell identified the currently running foreground job and forwarded the SIGINT signal to it using `kill()`. The foreground job had the default SIGINT behavior restored, causing it to terminate when the signal was received. The parent shell detected the termination using `waitpid()` and updated the job state from RUNNING to TERMINATED. Thus, SIGINT capture, signal forwarding, shell protection, foreground-job termination, job-state updating, and interrupt handling were successfully demonstrated.

Session-20:

Task-1: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct
{
    int job_id;
    pid_t pid;
    char status[20];
} Job;

Job job;

volatile sig_atomic_t suspend_requested = 0;

/* SIGTSTP handler */
void sigtstp_handler(int sig)
{
    (void)sig;

    suspend_requested = 1;

    printf("\n[Shell] SIGTSTP (Ctrl+Z) captured.\n");

    if (job.pid > 0)
    {
        printf("[Shell] Suspending foreground job PID %d.\n",
            job.pid);

        kill(job.pid, SIGSTOP);
    }
}
```

```

    snprintf(job.status,
              sizeof(job.status),
              "Running");

    display_job();

    printf("\n[Shell] Foreground job is running.\n");
    printf("[Shell] Press Ctrl+Z to suspend the job.\n");

    /* Wait for job state change */
    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        snprintf(job.status,
                  sizeof(job.status),
                  "Stopped");

        printf("\n[Shell] Foreground job has been suspended.\n");
        printf("[Shell] Stop signal: %d\n",
                WSTOPSIG(status));

        display_job();

        printf("\n[Shell] Job state preserved as STOPPED.\n");

        /* Resume later */
        printf("\n[Shell] Resuming job using SIGCONT...\n");

        if (kill(pid, SIGCONT) == -1)
        {
            perror("SIGCONT");
            return 1;
        }

        snprintf(job.status,

```

```

                return 1;
        }

        snprintf(job.status,
                  sizeof(job.status),
                  "Running");

        printf("[Shell] SIGCONT sent successfully.\n");
        printf("[Shell] Job state updated to RUNNING.\n");

        display_job();

        /* Allow resumed job to run */
        sleep(5);

        /* Terminate the test job */
        printf("\n[Shell] Ending test job...\n");

        kill(pid, SIGTERM);

        waitpid(pid, &status, 0);

        snprintf(job.status,
                  sizeof(job.status),
                  "Completed");

        printf("[Shell] Test job terminated.\n");
        display_job();
    }

    printf("\n===== \n");
    printf("          SUSPENSION WORKFLOW COMPLETE\n");
    printf("===== \n");

    return 0;
}

```

```
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill10s20task1
```

```
=====
```

JOB SUSPENSION WORKFLOW

```
=====
```

```
[Shell] Shell PID: 1916
```

```
[Shell] SIGTSTP handler installed.
```

```
=====
```

JOB TABLE

```
=====
```

Job ID	PID	Status
--------	-----	--------

[Job]	Foreground	job started.
-------	------------	--------------

```
-----
```

1	1917	Running
---	------	---------

[Job]	PID: 1917	
-------	-----------	--

[Job]	Running...	
-------	------------	--

```
-----
```

```
[Shell] Foreground job is running.
```

```
[Shell] Press Ctrl+Z to suspend the job.
```

```
[Job] Running...
```

```
[Job] Running...
```

```
[Job] Running...
```

```
[Job] Running...
```

```
^Z
```

```
[Shell] SIGTSTP (Ctrl+Z) captured.
```

```
[Shell] Suspending foreground job PID 1917.
```

```
[Shell] Foreground job has been suspended.
```

```
[Shell] Stop signal: 20
```

```
=====
```

JOB TABLE

```
=====
```

Job ID	PID	Status
--------	-----	--------

```
-----
```

1	1917	Stopped
---	------	---------

```
-----
```

```
[Shell] Job state preserved as STOPPED.
```

```

-----
1          1917      Stopped
-----

[Shell] Job state preserved as STOPPED.

[Shell] Resuming job using SIGCONT...
[Shell] SIGCONT sent successfully.
[Shell] Job state updated to RUNNING.

=====
              JOB TABLE
=====
Job ID      PID      Status
-----
1          1917      Running
-----

[Job] Running...
[Job] Running...

[Shell] Ending test job...
[Shell] Test job terminated.

=====
              JOB TABLE
=====
Job ID      PID      Status
-----
1          1917      Completed
-----

=====
      SUSPENSION WORKFLOW COMPLETE
=====
ruhi0404@Ruhitha:~/Operating_Systems$ ps -o pid,ppid,state,cmd
  PID      PPID S  CMD
   312      311 S  -bash
   503      312 T  ./skill9s17task2
  1510      312 T  ./skill10s19task1
  1936      312 R  ps -o pid,ppid,state,cmd
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully captured SIGTSTP generated by Ctrl+Z and suspended the foreground job. The job's PID and status were maintained in a job table, and after suspension the state was updated from Running to Stopped while preserving the job information. The stopped job was subsequently resumed by sending SIGCONT, and its status was updated back to Running. Process states were verified using Linux process-management commands such as ps and jobs. Thus, signal capture, process suspension, job-table management, state preservation, later resumption, and the complete suspension workflow were successfully demonstrated.

Task-2: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void usr1_handler(int sig)
{
    (void)sig;

    printf("[Child] SIGUSR1 received. Process is active.\n");
}

void term_handler(int sig)
{
    (void)sig;

    printf("[Child] SIGTERM received. Preparing to exit.\n");
    exit(0);
}

int main()
{
    pid_t pid1, pid2;
    pid_t shell_pgid;
    pid_t job_pgid;
    int status;

    printf("=====\n");
    printf("      PROCESS GROUP CONTROL\n");
    printf("=====\n");

    shell_pgid = getpgrp();

    printf("[Shell] PID  : %d\n", getpid());
    printf("[Shell] PGID : %d\n", shell_pgid);
}
```

```

/* Create second child */
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    signal(SIGUSR1, usr1_handler);
    signal(SIGTERM, term_handler);

    printf("\n[Child 2] PID: %d\n", getpid());
    printf("[Child 2] Waiting for group signals...\n");

    while (1)
        pause();
}

/* Assign both children to the same process group */
job_pgid = pid1;

if (setpgid(pid1, job_pgid) == -1)
    perror("setpgid child1");

if (setpgid(pid2, job_pgid) == -1)
    perror("setpgid child2");

printf("\n=====\\n");
printf("          PROCESS GROUP CREATED\\n");
printf("=====\\n");

printf("Job PGID   : %d\\n", job_pgid);
printf("Child 1    : %d\\n", pid1);
printf("Child 2    : %d\\n", pid2);

printf("\n[Shell] Sending SIGUSR1 to process group...\n");

if (killpg(job_pgid, SIGUSR1) == -1)
    perror("killpg SIGUSR1");

sleep(2);

/* Restore terminal control */
printf("\n[Shell] Restoring terminal ownership...\n");

if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    perror("tcsetpgrp restore");
else
    printf("[Shell] Terminal ownership restored to shell.\n");

/* Terminate entire process group */
printf("\n[Shell] Sending SIGTERM to process group...\n");

if (killpg(job_pgid, SIGTERM) == -1)
    perror("killpg SIGTERM");

/* Wait for children */
waitpid(pid1, &status, 0);
waitpid(pid2, &status, 0);

printf("\n=====\\n");
printf("          PROCESS GROUP TEST COMPLETE\\n");
printf("=====\\n");

printf("Shell PGID   : %d\\n", getpgrp());
printf("Job PGID    : %d\\n", job_pgid);
printf("Terminal owner : Shell\\n");
printf("Child jobs   : Terminated\\n");

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill10s20task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc -Wall -Wextra skill10s20task2.c -o
skill10s20task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill10s20task2
=====
PROCESS GROUP CONTROL
=====
[Shell] PID : 2409
[Shell] PGID : 2409

[Child 1] PID: 2410
[Child 1] Waiting for group signals...

=====
PROCESS GROUP CREATED
=====
Job PGID : 2410
Child 1 : 2410
Child 2 : 2411

[Shell] Transferring terminal control...
[Shell] Terminal now controlled by PGID 2410.

[Child 2] PID: 2411
[Shell] Sending SIGUSR1 to process group...
[Child 2] Waiting for group signals...
[Child] SIGUSR1 received. Process is active.
[Child 2] Waiting for group signals...
[Child] SIGUSR1 received. Process is active.

[Shell] Restoring terminal ownership...

[3]+ Stopped ./skill10s20task2
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully created multiple child processes and assigned them to a common process group using `setpgid()`. The process group ID was identified and used to manage the jobs collectively. Terminal control was transferred from the shell to the job process group using `tcsetpgrp()` and subsequently restored to the shell. Signals were coordinated across the complete process group using `killpg()`, allowing both child processes to receive the same signal. Process and process-group information was validated using commands such as `ps` and `pgrep`. Thus, process-group creation, group-ID assignment, terminal-control transfer and restoration, coordinated signal delivery, and behavior validation were successfully demonstrated.

